

Documentación — Módulo 4: Integral de Error (Regla del Trapecio)

Archivo: `modulo_4_integral_error.s`

1. Explicación del algoritmo implementado

El módulo implementa el cálculo de una **integral de error histórica** sobre los N datos de una columna de un archivo `.csv`, usando la **regla del trapecio** como método de integración numérica. El error se define como la desviación absoluta de cada lectura respecto a un valor ideal de referencia. La integral acumula esa desviación a lo largo del tiempo, produciendo un indicador de cuánto se ha alejado la variable monitoreada de su valor objetivo en el período analizado.

El algoritmo sigue tres pasos matemáticos (todo en enteros sin signo para las áreas):

1. Error puntual de cada lectura y lectura siguiente respecto al valor ideal:

```
ERROR_i = |Y_i - IDEAL|  
  
ERROR_(i+1) = |Y_(i+1) - IDEAL|
```

2. Área del trapecio formado por dos errores consecutivos (se asume intervalo de tiempo unitario entre muestras):

```
AREA_TRAPECIO = (ERROR_i + ERROR_(i+1)) / 2
```

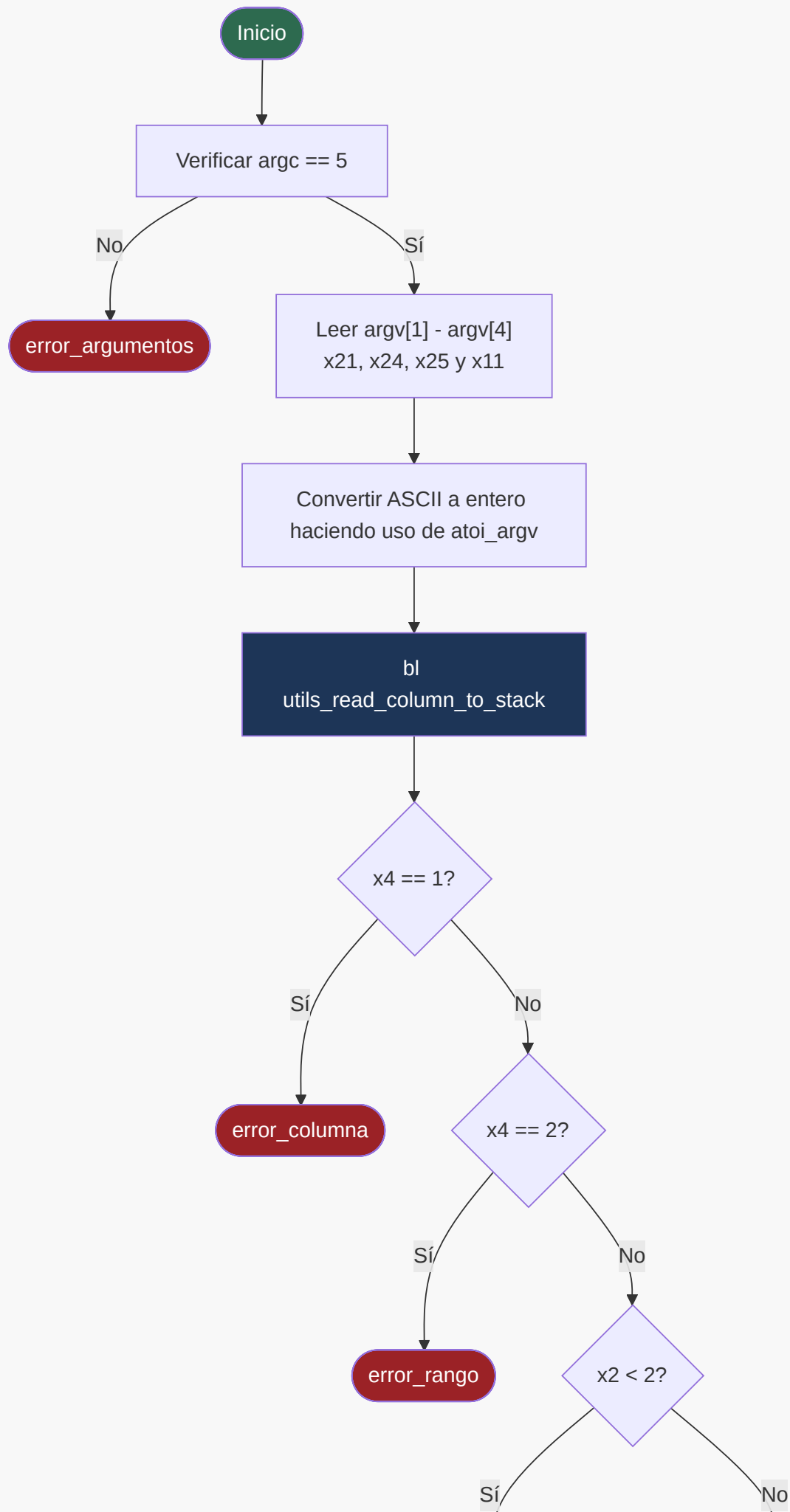
División entera, el resultado se trunca hacia cero.

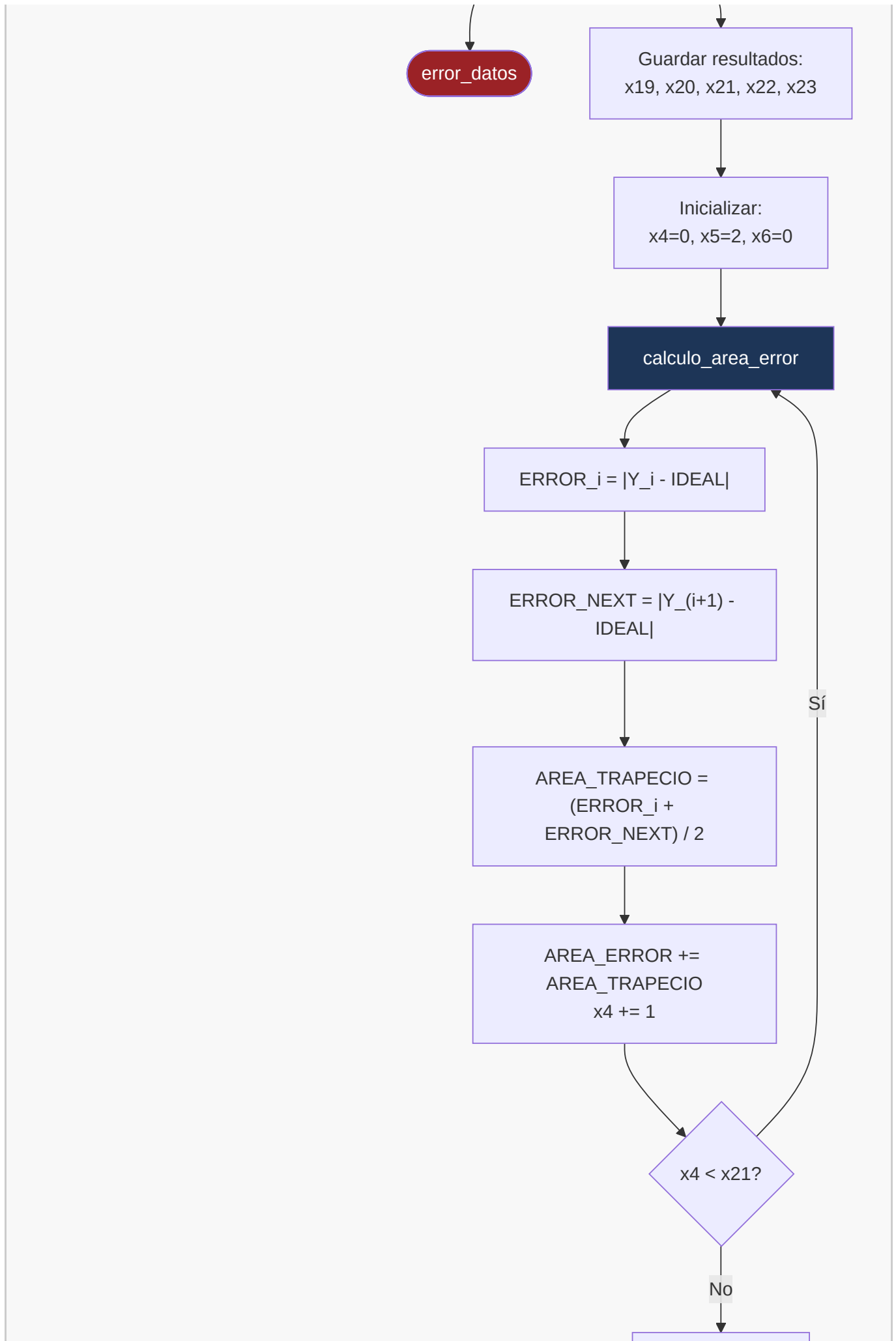
3. Acumulación de todas las áreas para obtener la integral total:

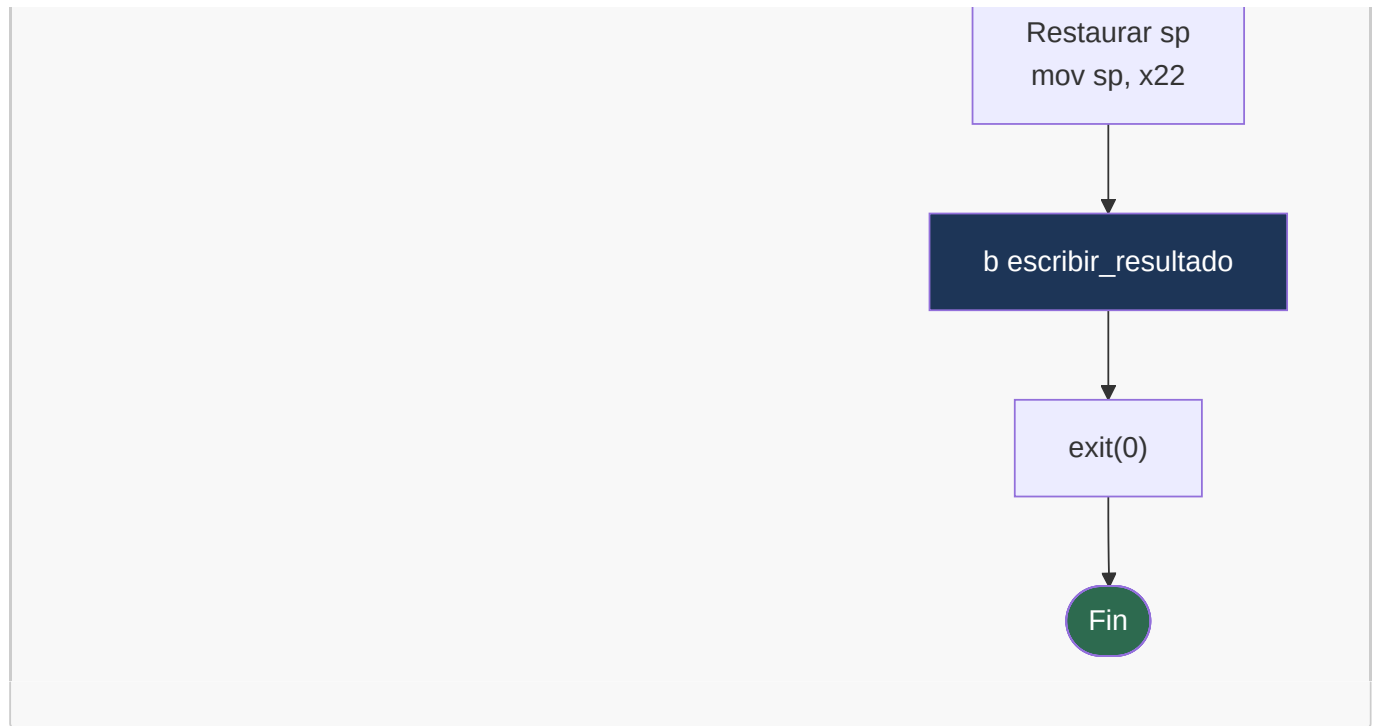
```
ERROR_INTEGRAL = Σ AREA_TRAPECIO para i = 0 .. N-2
```

El nombre del archivo a leer, la línea inicial, la línea final y el número de columna a analizar se reciben como argumentos (`argv[1]` - `argv[4]`). El módulo delega en `utils.s` la lectura del CSV y la extracción de los N valores de esa columna, realiza los cálculos y escribe el resultado en `resultado_integral.txt`.

2. Diagrama de flujo







Subrutina `escribir_resultado`

Inicio escribir_resultado

x26 = dirección de
buffer_resultado

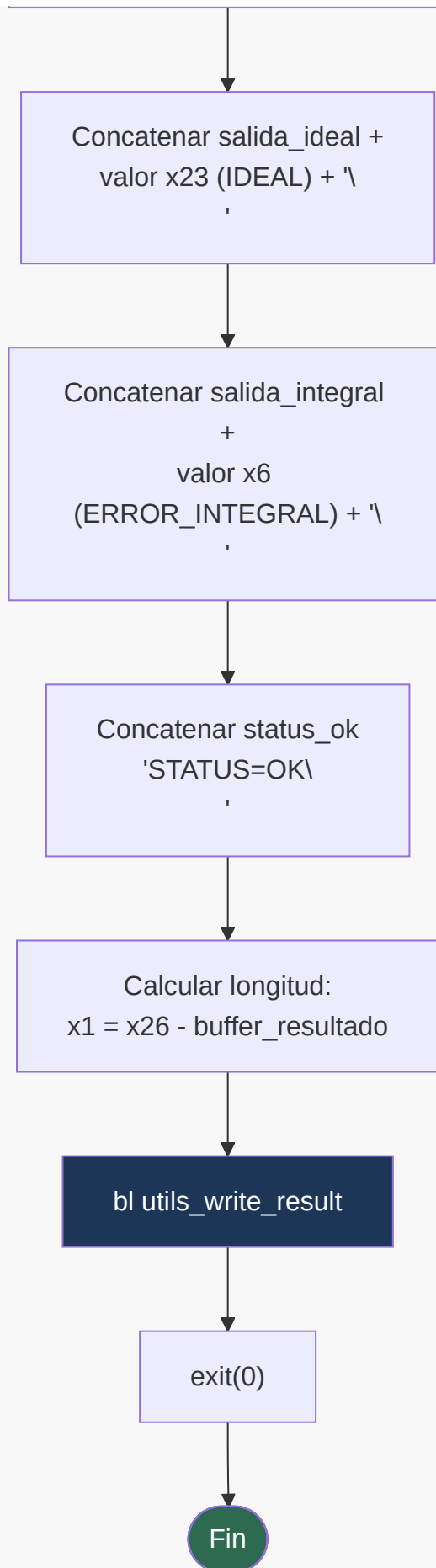
Concatenar salida_calculo
'CALC=ERROR_INTEGRAL\
,

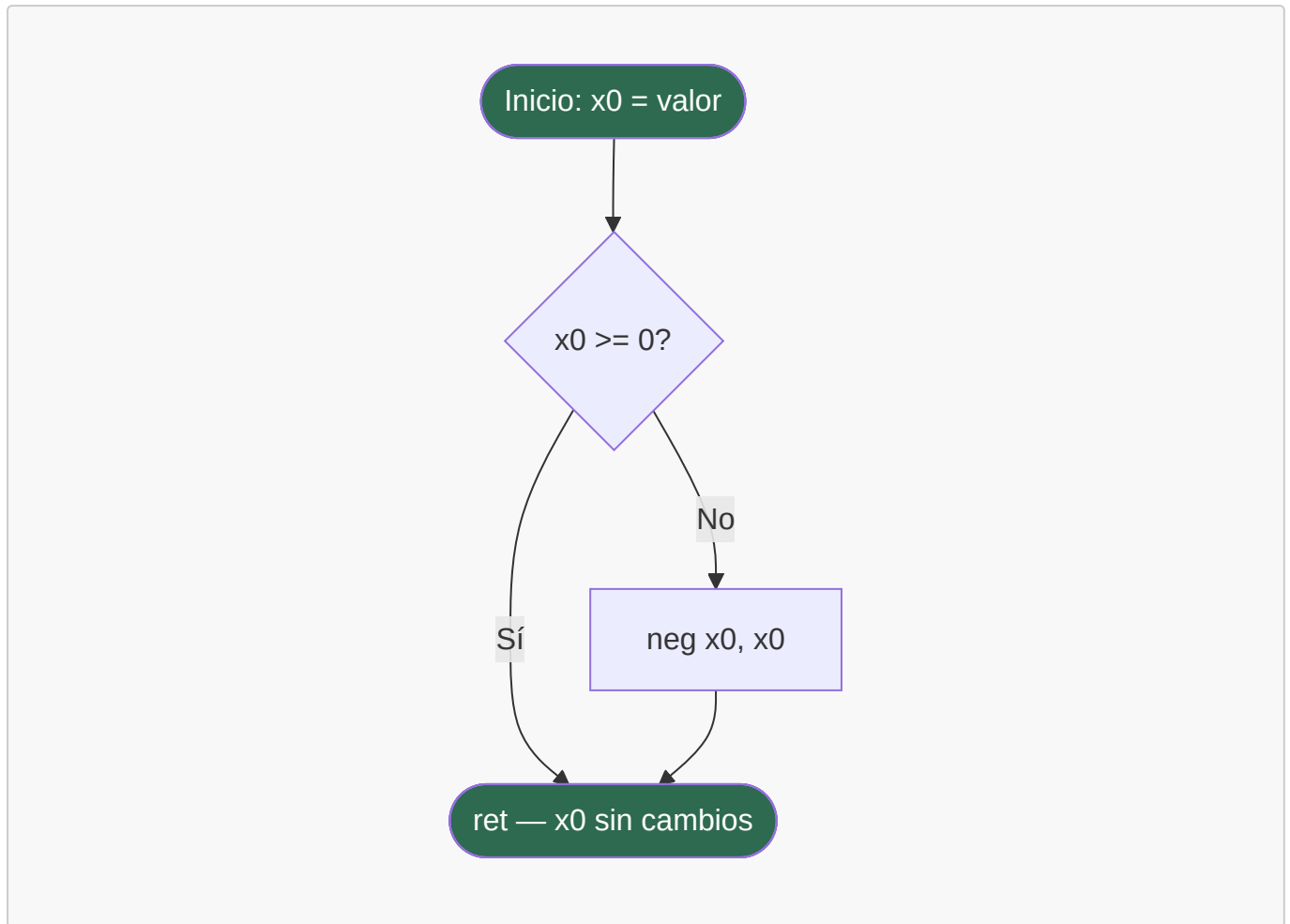
Concatenar
salida_columna +
valor x27 + '\
,

Concatenar salida_inicio +
valor x24
(WINDOW_START) + '\
,

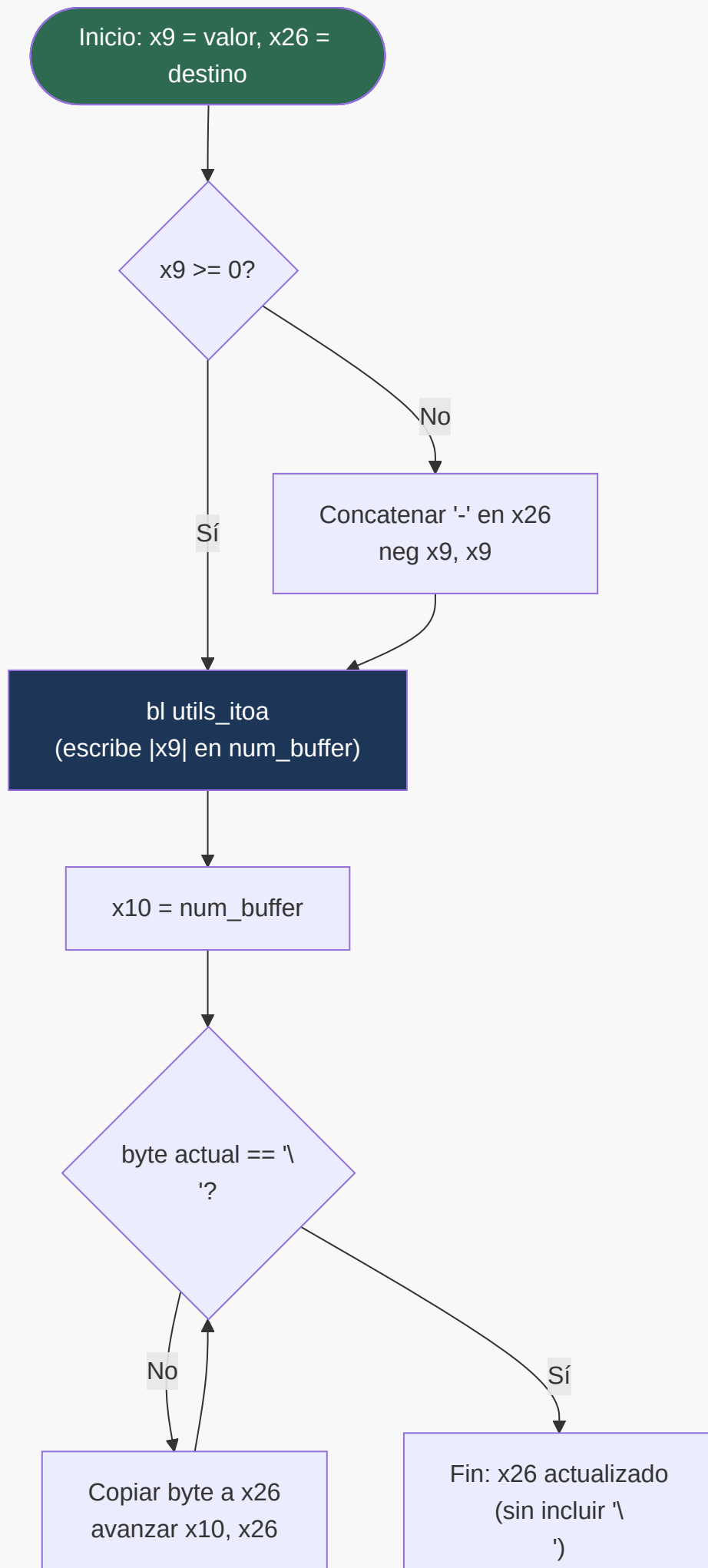
Concatenar salida_final +
valor x25 (WINDOW_END)
+ '\
,

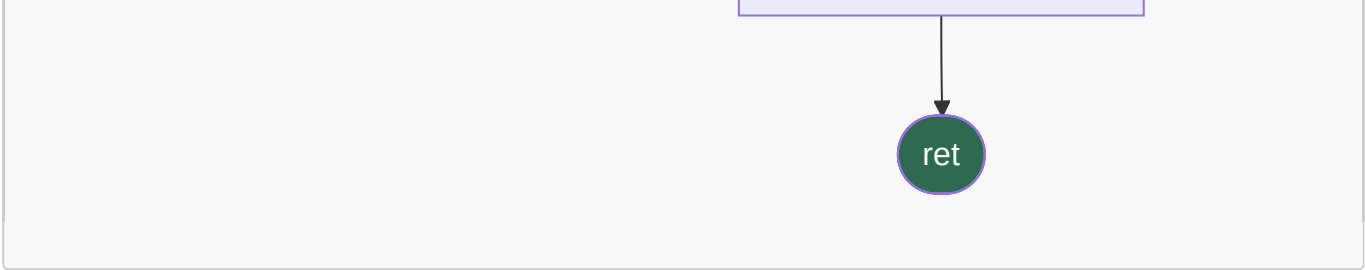
Concatenar
salida_contador +
valor x20 (COUNT) + '\
,

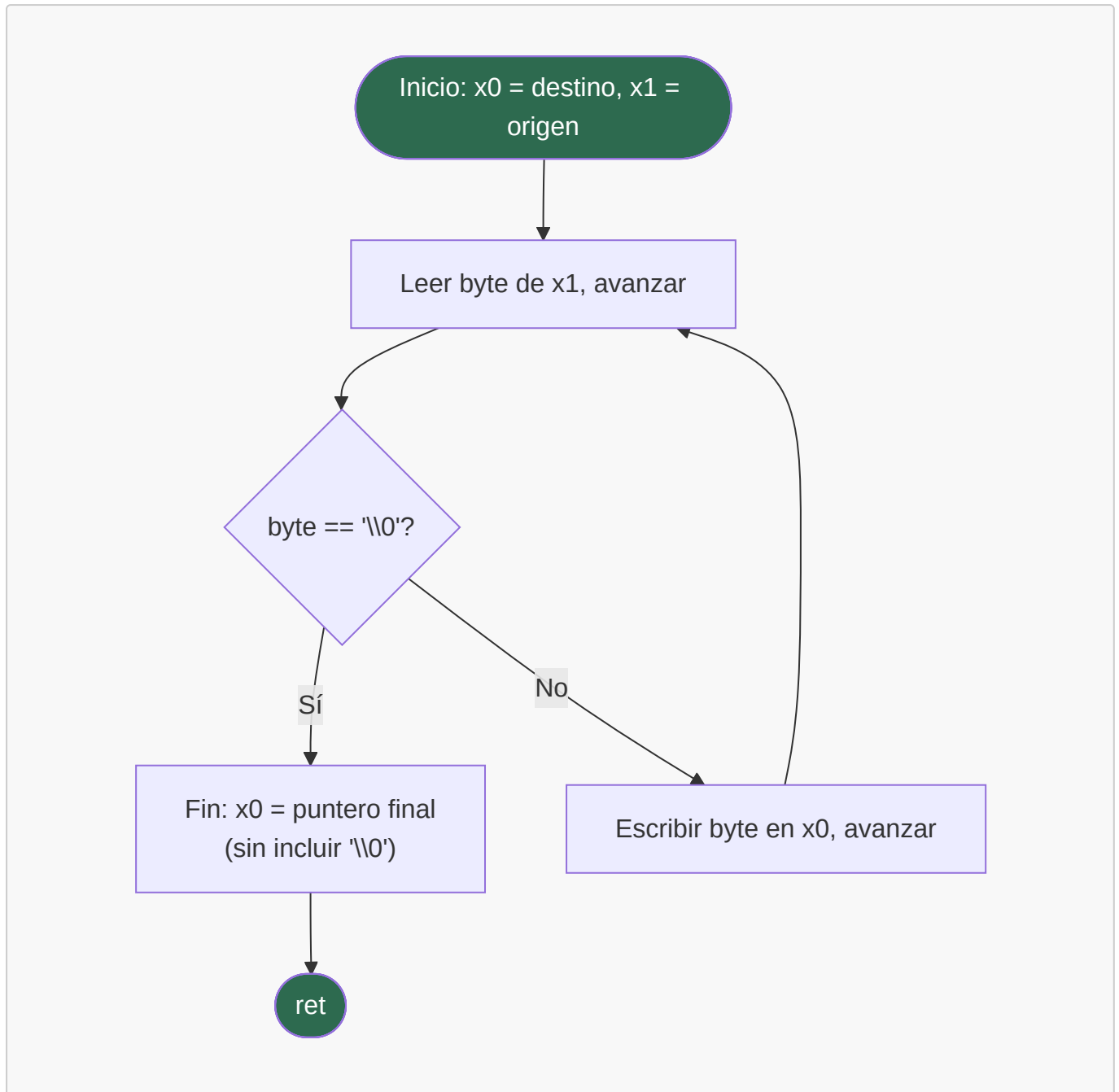


Subrutina **valor_absoluto**

Subrutina `utilidad_escribir_entero`





Subrutina `utilidad_concatenar_cadena`

3. Explicación de los registros utilizados

Registro	Propósito
x11	Número de columna a leer (parámetro de entrada para <code>utils_read_column_to_stack</code>), obtenido de <code>argv[4]</code> .
x19	Dirección base del área de datos en el stack devuelta por <code>utils.s</code> . Ajustada con <code>sub x19, x19, #16</code> para apuntar al dato más reciente; los datos se acceden con desplazamientos negativos crecientes.
x20	Cantidad de números guardados (x2 de <code>utils.s</code>).
x21	N - 1: número de trapecios a calcular, usado como límite del ciclo.
x22	Valor de <code>sp</code> original a restaurar al finalizar (x3 de <code>utils.s</code>), para dejar la pila balanceada.
x23	Valor IDEAL de referencia, determinado por el número de columna a analizar.
x24	Línea inicial del rango a leer (<code>argv[2]</code>).
x25	Línea final del rango a leer (<code>argv[3]</code>).
x27	Número de columna leída (sensor), guardado desde x11 para usarlo en la sección de escritura de resultados.
x4	Índice <code>i</code> del ciclo de integración, inicializado en 0.
x5	Divisor constante 2, usado en el cálculo de cada <code>AREA_TRAPECIO</code> .
x6	Acumulador de <code>AREA_ERROR</code> (resultado final de la integral). Antes de <code>utils_read_column_to_stack</code> almacena temporalmente el valor ideal.
x26	Puntero dinámico que recorre <code>buffer_resultado</code> mientras se construye el reporte de texto; se actualiza después de cada concatenación.
x9	Valor entero (con signo) que se pasa a <code>utilidad_escribir_entero</code> para convertir y escribir.
x0-x3, x7-x9, x10, x12	Registros de propósito general usados dentro del ciclo y las subrutinas auxiliares para punteros, contadores y caracteres temporales (no preservados entre llamadas, de "uso libre" según convención).

4. Explicación de ciclos, saltos y subrutinas

Validación de argumentos

Lo primero que hace `_start` es comprobar que `argc == 5` (programa + archivo + inicio + fin + columna + ideal). Si no se cumple, salta a `error_argumentos` antes de leer cualquier argumento:

```
ldr x0, [sp]
cmp x0, #6
```

bne error_argumentos

Lectura de argumentos y conversión ASCII a entero

Se leen `argv[1]` (puntero al nombre del archivo), `argv[2]` (línea inicial), `argv[3]` (línea final) y `argv[4]` (columna) mediante `atoi_argv`. El resultado numérico queda en `x10` y se copia al registro correspondiente. Finalmente llama a `utils_read_column_to_stack`.

bl utils_read_column_to_stack (subrutina externa)

Llamada única que delega en `utils.s` toda la apertura y lectura del CSV y el parsing de la columna solicitada. Devuelve en `x0-x3` los punteros al área de datos en el stack, y en `x4` un código de error (1 = columna inválida, 2 = rango inválido). Las comprobaciones de error se realizan inmediatamente tras el retorno, antes de usar cualquier valor devuelto.

Ciclo calculo_area_error

Núcleo del módulo. Itera sobre los `N - 1` pares de datos consecutivos calculando el área de cada trapecio y acumulándola en `x6`:

```
calculo_area_error:
    lsl x2, x4, #4           // offset_i = i * 16
    sub x2, x19, x2
    ldr x0, [x2]             // Y_i
    sub x0, x0, x23
    bl valor_absoluto        // x1 = ERROR_i

    add x2, x4, #1
    lsl x2, x2, #4           // offset_(i+1) = (i+1) * 16
    sub x2, x19, x2
    ldr x0, [x2]             // Y_(i+1)
    sub x0, x0, x23
    bl valor_absoluto        // x0 = ERROR_NEXT

    add x9, x1, x0
    udiv x9, x9, x5           // AREA_TRAPECIO = (ERROR_i + ERROR_NEXT) / 2
    add x6, x6, x9           // AREA_ERROR += AREA_TRAPECIO

    add x4, x4, #1
    cmp x4, x21
    blt calculo_area_error   // repetir mientras i < N-1
```

El desplazamiento usa `lsl #4` (multiplicación por 16) porque cada elemento en el stack ocupa 16 bytes. Los datos se acceden con desplazamiento negativo desde `x19` (`sub x2, x19, offset`) porque el stack crece hacia abajo y los datos fueron apilados en orden de lectura.

valor_absoluto (subrutina sin ciclo)

Subrutina de propósito único que garantiza que el error siempre sea no negativo antes de sumarlo al trapecio:

```
valor_absoluto:
    cmp x0, #0
    bge num_positivo    // salto condicional: si x0 >= 0, no hacer nada
    neg x0, x0
num_positivo:
    ret
```

escribir_resultado (secuencia lineal sin ciclos propios)

Construye el reporte concatenando en orden los bloques del formato de salida. Cada bloque sigue el mismo patrón:

```
mov x0, x26
ldr x1, =etiqueta
bl  utilidad_concatenar_cadena    // copia la etiqueta, devuelve nueva
posición
mov x26, x0
mov x9, <registro_con_valor>
bl  utilidad_escribir_entero      // escribe el número, actualiza x26
mov x0, x26
ldr x1, =salto_linea
bl  utilidad_concatenar_cadena    // agrega '\n'
mov x26, x0
```

Al final calcula la longitud total (`x26 - dirección_inicial_del_buffer`) y llama a `utils_write_result` para escribir el archivo.

utilidad_escribir_entero (subrutina con ciclo de copia)

Maneja el signo manualmente porque `utils_itoa` solo soporta enteros positivos:

```

cmp x9, #0
bge .ue_positivo      // salto condicional: si x9 >= 0, omitir el signo
... escribir '-' y neg x9, x9 ...
.ue_positivo:
    bl utils_itoa      // convierte |x9| a "digitos\n\0" en num_buffer
    ldr x10, =num_buffer
.ue_copy:              // ciclo: copia carácter por carácter
    ldrb w12, [x10], #1
    cmp w12, #10        // ¿es '\n'?
    beq .ue_fin         // salto condicional: termina sin copiar el '\n'
    strb w12, [x26], #1
    b .ue_copy          // salto incondicional: repetir
.ue_fin:

```

El ciclo `.ue_copy` recorre el resultado de `utils_itoa` y lo transfiere al buffer de salida, **deteniéndose antes del `\n`** que `utils_itoa` siempre agrega, porque el formato `CLAVE=VALOR\n` ya añade su propio salto de línea por separado.

utilidad_concatenar_cadena (subrutina con ciclo de copia)

```

.loop_copia_bytes:
    ldrb w2, [x1], #1      // lee byte de origen, avanza
    cbz w2, .fin_copia_bytes // salto condicional: si es '\0', termina
    strb w2, [x0], #1      // escribe byte en destino, avanza
    b .loop_copia_bytes    // salto incondicional: repetir
.fin_copia_bytes:
    ret

```

Ciclo clásico de copia de string terminado en `\0`, usado para transferir las etiquetas estáticas y los caracteres especiales al buffer dinámico de salida.

Bloque de errores

Cada etiqueta de error (`error_argumentos`, `error_ideal`, `error_columna`, `error_rango`, `error_datos`) sigue la misma estructura: inicializar `x26` con la dirección de `buffer_resultado`, concatenar `STATUS=ERROR`, el campo `ERROR=` con su valor, el campo `DETAIL=` con su detalle, calcular longitud, escribir a `errores_integral.txt` y terminar con `exit(0)`.

5. Manejo de errores

Error	Condición	Archivo de salida
INSUFFICIENT_ARGUMENTS	argc != 5	errores_integral.txt
INVALID_COLUMN	x4 == 1 tras utils_read_column_to_stack	errores_integral.txt

Error	Condición	Archivo de salida
INVALID_RANGE	x4 == 2 tras utils_read_column_to_stack	errores_integral.txt
INSUFFICIENT_DATA	x2 < 2 (menos de 2 valores leídos)	errores_integral.txt

El formato de salida de error es:

```
STATUS=ERROR
ERROR=<TIPO_ERROR>
DETAIL=<DETALLE>
```

6. Explicación del formato de entrada y salida

Entrada

- **Argumento `argv[1]`:** nombre del archivo a leer.
- **Argumento `argv[2]`:** línea inicial del rango a leer.
- **Argumento `argv[3]`:** línea final del rango a leer.
- **Argumento `argv[4]`:** número de columna del sensor a analizar (2–7).
- **Formato del CSV:** encabezado seguido de filas de 9 columnas separadas por comas, terminado en \$:

```
ID,TEMP,HUM_AIRE,HUM_SUELO_1,HUM_SUELO_2,LUZ,GAS,RIEGO_1,RIEGO_2
1,31,65,60,18,527,50,0,0
...
30,27,74,58,15,347,91,0,0
$
```

Salida

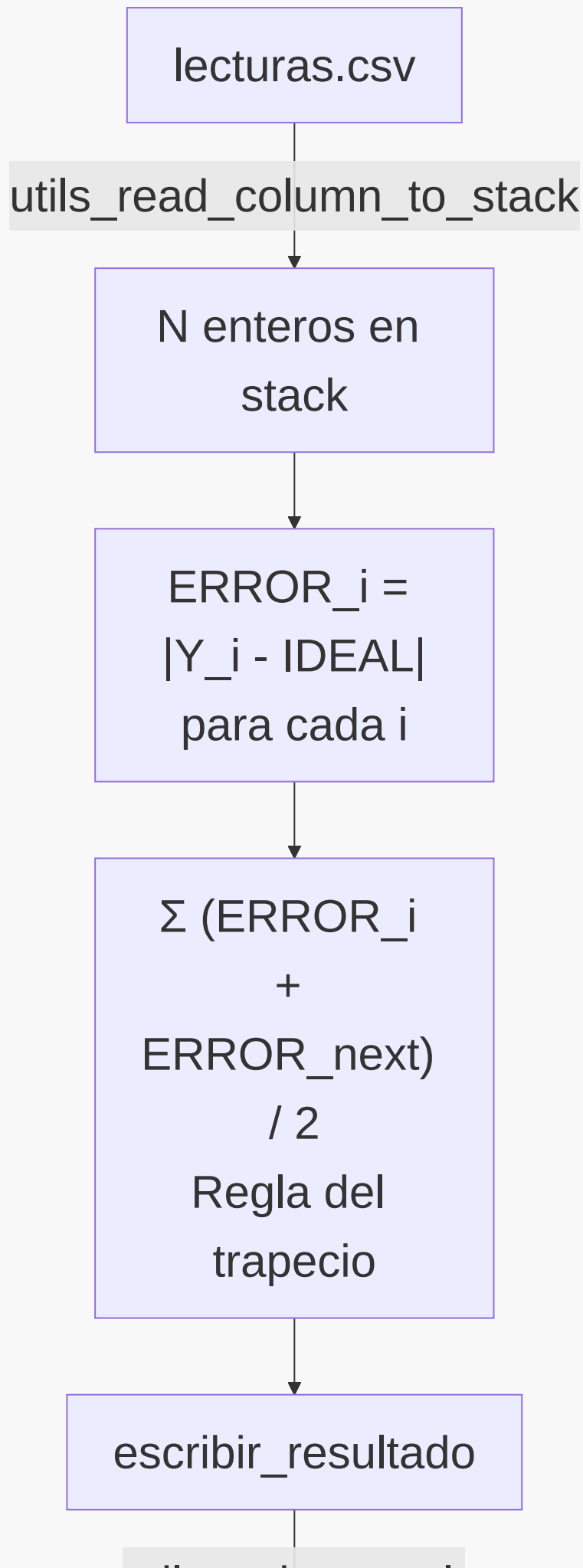
- **Archivo:** `resultado_integral.txt`, texto plano en formato `CLAVE=VALOR`, una por línea, leído posteriormente por el backend para almacenarse en MongoDB y mostrarse en el dashboard.
- **Formato de salida exitosa:**

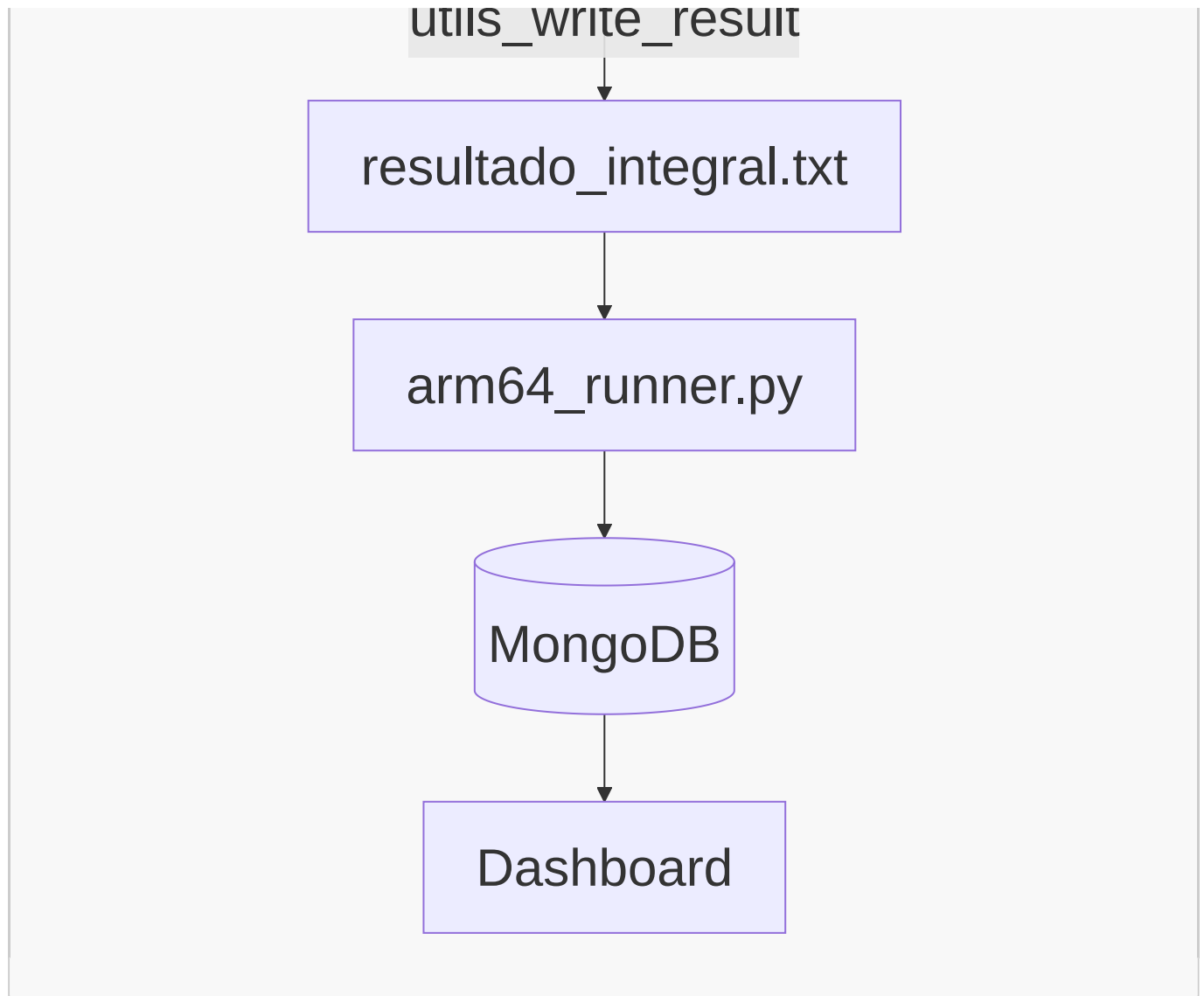
```
CALC=ERROR_INTEGRAL
COLUMN=<número_de_columna>
WINDOW_START=<línea_inicial>
WINDOW_END=<línea_final>
COUNT=<cantidad_de_datos>
IDEAL=<valor_ideal>
```

```
ERROR_INTEGRAL=<resultado>  
STATUS=OK
```

- Todos los valores son **enteros**; **ERROR_INTEGRAL** es siempre no negativo al ser suma de valores absolutos.

7. Flujo general resumido





8. Nota sobre manejo de memoria

El módulo no reserva memoria propia para los datos: reutiliza directamente el área del stack que `utils_read_column_to_stack` deja apilada, y restaura `sp` (`mov sp, x22`) al terminar el ciclo de integración, dejando la pila balanceada antes de construir el reporte.

La única memoria propia declarada está en `.bss`:

Buffer	Tamaño	Uso
<code>num_buffer</code>	32 bytes	Conversión temporal de enteros a texto (<code>utils_itoa</code>)
<code>buffer_resultado</code>	512 bytes	Reporte completo antes de escribirlo a disco

9. Depuración GDB

Se está evaluando el comportamiento de los registros usados para evitar que se manejen valores erróneos durante los cálculos

```

12  start
13
14 //Comprobo que Row=2
15 cmp x2, #2
16 btl error_datos
17
18 sub x19, x19, #10
19 sub x21, x20, #1
20
21 mov x4, #0
22 //Indice i inicializado en 0
23 mov x5, #2
24 //Divisor de AREA TRAFICO
25 mov x6, #0
26
27 calcula_area_error:
28 //array[i] = direccion base + (i * tam_bytes)
29 //array[i] = x19 + (i4 * 8)
30 //ERROR_1 = (Y-1) * IDEAL
31 lsl x2, x4, #8
32 sub x2, x19, x2
33 ldr x6, [x2]
34 sub x5, x5, x23
35 //x6 = Y-1
36 btl valor_absolute
37 mov x1, x6
38
39 //array[i+2] = direccion base + ((i+2) * tam_bytes)
40 //array[i+2] = x19 + ((i4 + 1) * 8)
41 //ERROR_NEXT = (Y-(i+1)) * IDEAL
42 mov x2, x4, #8
43 lsl x2, x2, #8
44 sub x2, x19, x2
45 ldr x6, [x2]
46 sub x5, x5, x23
47 //x6 = Y-(i+1)
48 btl valor_absolute
49 sub x6, x6, x23
50 //x6 = ERROR_NEXT
51
52 //Fin de la funcion
53 ret

```